Conversation notifications for Xpresso

Implementation guide for the developer taking this over (junior or AI)

Version	1.0 - 16 October 2026
Prepared for	Willy (project owner)
Platform	Android only. Capacitor app wrapping a Svelte 5 front end; built with AI Studio
Push	Firebase Cloud Messaging (FCM), HTTP v1
Deliverable	Local Capacitor plugin **capacitor-conversation-notifications** (folder delivered alongside this PDF)

1. Objective

Today an Xpresso message appears in the Android notification shade under *Notifications avec alerte* (plain alerts), while WhatsApp messages appear under *Conversations*. The goal is for Xpresso chat messages to appear under **Conversations**. A second defect is visible in the same screenshot: the notification body shows raw colour markup (for example `\color[#1d4ed8]{Hello}`). The notification must show only the text (**Hello**).

Two requirements, in order of priority:

1. Chat notifications are classified by Android as conversations, with sender name, avatar and message history.
2. Colour markup never reaches the user in a notification: only the visible text is shown.

2. How Android decides what is a conversation

Since Android 11, a notification is placed in the Conversations section only if **all** of the following are true. Capacitor's stock notification plugins satisfy none of them, which is why Xpresso is shown as a plain alert.

Requirement	What it means in code
Messaging style	Built with `NotificationCompat.MessagingStyle` and at least one `Person` (the sender).
Conversation shortcut	A **long-lived** shortcut (`setLongLived(true)`) for the chat is published with `ShortcutManagerCompat`, and the notification references it with `setShortcutId(chatId)`. Shortcut id and shortcut reference must be identical.
Message category and importance	`CATEGORY_MESSAGE` on a channel with importance HIGH.

Samsung One UI follows the same rules. Android 10 and older have no Conversations section; there the notification still renders in message style.

3. Architecture

The notification must be built **natively**, because it has to work when the app is closed and the Svelte layer is not running.

Step	Where	What happens
1	Server	Sends an FCM **data-only** message (no `notification` key) with the payload in section 6. The text is already converted to plain text with `toPlainText()`.
2	Android, native	`ChatMessagingService` receives it. It wins over Capacitor's own service because its manifest priority is 100 (Capacitor's is 0). Only one service receives each FCM message.
3	Android, native	`TextSanitizer` strips colour markup again (safety net). `ChatNotifier` publishes the shortcut and the MessagingStyle notification, stacking messages of the same chat.
4	Android, native	The same message is also forwarded to Capacitor's push plugin (by reflection) so the web layer still gets `pushNotificationReceived` when the app is open; token refreshes are forwarded too.
5	Svelte	Registers the chat on screen, pre-creates shortcuts, and reacts to notification taps (cold start and warm start).

4. What has been delivered

File (inside the plugin folder)	Role
`android/src/main/AndroidManifest.xml`	Registers `ChatMessagingService` with priority 100.
`android/.../ChatMessagingService.java`	Receives FCM, validates payload, sanitizes text, shows notification, forwards to Capacitor.
`android/.../ChatNotifier.java`	Channel, shortcut, MessagingStyle, avatar download (3 s timeouts, https only) with letter-avatar fallback, message stacking.
`android/.../TextSanitizer.java`	Colour markup to plain text. Fully unit-tested (see section 8).
`android/.../ConversationNotificationsPlugin.java`	JS bridge: `upsertShortcut, setActiveChat, clearChat, clearAll, getPendingChat` and the `chatOpened` event.
`android/build.gradle`	Library module; uses the app's own Gradle versions when present.
`js/conversationNotifications.ts`	Typed bridge. Copy into the app (for example `src/lib/native/`).
`js/sanitize.ts`	Same sanitizer for the server and the app. Copy into both.
`js/App.example.svelte`	Svelte 5 wiring example.

5. Implementation steps

Do the steps in order. Do not edit files inside the generated `android/` folder: it can be regenerated and the edits lost. All native code lives in the plugin.

Step 1 - Install the plugin

```
# 1. Place the folder in the project, for example: ./plugins/capacitor-conversation-notifications
# 2. From the project root:
npm i ./plugins/capacitor-conversation-notifications
npx cap sync android
```

Check that `android/capacitor.settings.gradle` now lists the plugin. Keep `@capacitor/push-notifications` installed: it is still used for `requestPermissions()` (Android 13+ notification permission) and `register()`.

Step 2 - Align versions if the Android build complains

The plugin reads `compileSdkVersion, minSdkVersion, targetSdkVersion, androidxCoreVersion, firebaseMessagingVersion` from the app's `variables.gradle` when defined, otherwise uses its own defaults (compileSdk 35, minSdk 23, Java 17). If the build fails on a version mismatch, change the defaults in the plugin's `build.gradle` to match the app.

Step 3 - Change the server to send data-only messages

Remove the `notification` block from the FCM request and put everything in `data` (values must be strings). Set `android.priority` to HIGH so delivery is immediate. See the payload contract in section 6. Convert the text with `toPlainText()` from `js/sanitize.ts` before sending.

Step 4 - Wire the Svelte layer

Copy the two TypeScript files into the app and mount the logic from `App.example.svelte` once at the root:

- Call `upsertShortcut` for every chat when the chat list loads or changes.
- On the chat screen call `setActiveChat({ chatId })` when it opens and `setActiveChat({ chatId: null })` when it closes. No notification is shown for the chat that is open, and its existing notification is dismissed.
- On startup call `getPendingChat()` and navigate if it returns a chat id (app was closed when the notification was tapped).
- Listen to `chatOpened` and navigate (app was already running).
- On logout call `clearAll()`.

Step 5 - Optional: your own status-bar icon

A white chat bubble is used by default. To use the Xpresso mark, add a white-on-transparent drawable and declare it in the app manifest:

```
<meta-data android:name="com.google.firebase.messaging.default_notification_icon"
           android:resource="@drawable/ic_stat_xpresso" />
```

6. FCM payload contract

Every value is a string. Messages that do not have `type = chat_message` and a `chatId` are not turned into conversation notifications; they are passed to Capacitor's plugin untouched.

Key	Required	Description
`type`	yes	`chat_message`
`chatId`	yes	Stable id of the conversation. Also used as the shortcut id and to derive the notification id, so the same chat always updates the same notification.
`senderId`	yes	Stable id of the sender.
`senderName`	yes	Display name (colour markup is stripped).
`text`	yes	Message text. Plain text preferred; colour markup is stripped on the phone as a safety net.
`ts`	no	Epoch milliseconds as a string. Defaults to arrival time.
`avatarUrl`	no	https URL of the sender picture. A coloured letter avatar is used when absent or unreachable.
`isGroup`	no	`true` for group chats; then also send `chatTitle`.

Key	Required	Description
`chatTitle`	no	Group name (groups only).
`fallbackText`	no	Shown when `text` is empty after stripping, for example a media-only message. Default: New message.

Example request body (FCM HTTP v1):

```
{
  "message": {
    "token": "<device token>",
    "data": {
      "type": "chat_message",
      "chatId": "c_4821",
      "senderId": "u_77",
      "senderName": "Bosco",
      "text": "Hello",
      "ts": "1760608380000",
      "avatarUrl": "https://example.com/avatars/u_77.jpg",
      "isGroup": "false"
    },
    "android": { "priority": "HIGH" }
  }
}
```

7. Colour markup rules

Messages are stored with colour markup so the chat can render coloured text. A notification must show only the visible text. The sanitizer removes the colour wrapper and keeps what is inside it, including nested wrappers and braces that belong to the text.

Input	Notification shows
`\color[#1d4ed8]{Hello}`	Hello
`\color[#1d4ed8]{Je l,aime!}`	Je l,aime!
`Hi \color[#f00]{red} and \color[#0f0]{green}!`	Hi red and green!
`\color[#111]{a \color[#222]{b} c}`	a b c
`\color{red}{x}`, `\textcolor{red}{x}`, `\textcolor[HTML]{FF0000}{x}`	x
`\color[#123]{a {b} c}`	a {b} c
`\color[#123]{unfinished`	unfinished
`plain {braces} text` and `\colorbox{x}`	unchanged (not colour markup)

Rules for the developer:

- Sanitize in **two** places: the server (so the payload is already clean) and the phone (safety net). The Java class and the TypeScript file implement the same algorithm and must stay in sync.
- Sanitize `senderName` and `chatTitle` as well as `text`.
- Do not strip other backslash commands in general: text such as a file path would be damaged. If a new markup form appears, add it to both implementations and to the test cases.
- If the text is empty after stripping, show `fallbackText` or New message; never an empty notification.

8. Testing

8.1 Sanitizer (already done)

18 cases covering the table above, unterminated markup, nested markup, escaped braces, empty and null input pass in both the Java and the TypeScript implementation. When you change either file, rerun the same cases.

8.2 On a real Android 11+ device (not an emulator image without Google Play services)

1. Install a debug build, log in, grant the notification permission.
2. Send a data-only message with the example body above using curl:

```
curl -X POST "https://fcm.googleapis.com/v1/projects/<PROJECT_ID>/messages:send" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  -d @payload.json
```

3. Put the app in the background, then fully close it (swipe away) and send again. The notification must still appear.
4. Pull down the shade: the notification must be under **Conversations**, with the sender name, avatar and the text **Hello** (no markup).
5. Send three messages in a row from the same chat: they stack in one notification.
6. Send a message with text `\color[#1d4ed8]{Hello}` (bypassing the server sanitizer): the phone must still show only Hello.
7. Open that chat: its notification disappears; send another message while it is open: no notification is shown.
8. Tap a notification with the app closed and with the app open: both land on the right chat.

8.3 Inspecting what Android stored

```
adb shell dumpsys notification --noredact | grep -B2 -A12 "<your.app.id>"
adb shell dumpsys shortcut | grep -A6 "<your.app.id>"
```

The notification should list a shortcut id equal to the chat id and a MessagingStyle; the shortcut dump should list the same id as long-lived.

9. Troubleshooting

Symptom	Likely cause and fix
Still under alerts, not Conversations	The server still sends a `notification` block, so the system shows it and the service never runs. Switch to data-only. Also confirm the shortcut id equals the notification's shortcut id (both are the chat id).
No notification at all when the app is closed	FCM priority is not HIGH, battery optimisation on the device, or the notification permission (Android 13+) was not granted.
Capacitor `pushNotificationReceived` stopped firing	Reflection target not found. Check the Logcat tag ConvNotif; confirm the class name in `CAP_PUSH` matches the installed push plugin version, or that a different push plugin is not in use.
Token not reaching the server	Same cause: `onNewToken` forwarding failed. Fix the forward target as above.
Plain white square instead of an icon	Provide a white-on-transparent drawable and declare `default_notification_icon` (Step 5).
Avatar is a coloured letter	Normal when `avatarUrl` is missing, not https, or slow (3 s timeout). The message is still delivered.
Build fails after `cap sync`	Align Gradle and SDK versions (Step 2).
Colour markup still visible	A markup form not yet supported. Add it to `TextSanitizer.java` and `sanitize.ts` with a test case.

10. Acceptance criteria

- Xpresso messages appear under Conversations on a real Android 11+ device, with the app open, in the background and closed.
- The notification text never contains colour markup.
- Repeated messages of one chat stack in a single notification; different chats stay separate.
- No notification for the chat currently on screen; opening a chat clears its notification.
- Tapping a notification opens the correct chat in every app state.
- Non-chat FCM messages and token refreshes still reach Capacitor's push plugin as before.

Known limits. The native Android classes have not been compiled in the environment where this guide was written (no Android SDK). Run a debug build and the device checklist above before release. Android may demote conversations the user never interacts with, and the user can move a conversation out of the section; neither is under the app's control.